



Nome do Integrante (ordem alfabética)	RA
Edmilson Li Quansang	10425514
João Paulo Berto Massabki	10425593
Pietro Caffettani	10425628

Tendo por base a linguagem de programação a ser utilizada no desenvolvimento do seu projeto da disciplina, faça os exercícios a seguir.

Matriz de Adjacência – Grafos Direcionados e Não direcionados

- 1) Escreva um método “int inDegree(int v)” que calcula e retorna o grau de entrada de um vértice **v** de um grafo dirigido. O método deve ser implementado na classe TGrafo da matriz de adjacência. Obs.: Grau de entrada de v é o total de arestas que chegam no vértice v.
- 2) Escreva o método outDegree(int v) que calcula o grau de saída de **v** em grafo dirigido. O método deve ser implementado na classe TGrafo que usa matriz de adjacência. Obs.: Grau de saída de v é o total de arestas que saem do vértice v.
- 3) Fazer o método degree(int v) que calcula o grau do vértice de um grafo dirigido. O método deve ser implementado na classe TGrafo que usa matriz de adjacência.
- 4) Escreva um método para um grafo direcionado que recebe um vértice como parâmetro e retorne 1 se vértice for uma **fonte** (grau de saída maior que zero e grau de entrada igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe TGrafo como matriz de adjacência.
- 5) Escreva um método para um grafo direcionado que recebe um vértice como parâmetro, retorne 1 se vértice for um **sorvedouro** (grau de entrada maior que zero e grau de saída igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe TGrafo que utiliza matriz de adjacência.
- 6) Escreva um método que receba um **grafo dirigido** como parâmetro e **retorna 1** se o grafo for **simétrico** e 0 caso contrário. O método deve ser implementado para a classe TGrafo que utiliza matriz de adjacência.
- 7) Um **grafo** pode ser armazenado em um arquivo com o seguinte formato:

```
6
8
0 1
0 5
1 0
1 5
2 4
3 1
4 3
3 5
```

Onde na primeira linha contém um inteiro **V** (vértice), na segunda contém um inteiro **A** (arestas) e nas demais linha contém dois inteiros pertencentes ao intervalo 0..V-



Teoria dos Grafos

1. Se interpretarmos cada linha do arquivo como uma **aresta**, podemos dizer que o arquivo define um **grafo** com vértices 0..V-1. Escreva um método que receba um nome de arquivo com o formato acima e construa a representação do grafo como matriz de adjacência. (Exemplo de código que manipula arquivo no próximo slide)
- 8) Criar uma outra classe TGrafoND e modifique as funções insereA, removeA e show para representar um **grafo não-dirigido** utilizando matriz de adjacência.
- 9) Fazer o método degree(int v) que calcula o grau do vértice de um grafo não-dirigido. O método deve ser implementado na classe TGrafoND que usa matriz de adjacência.
- 10) Modifique a classe TGrafo e os métodos correspondentes para permitir a criação de um grafo direcionado rotulado (valor float) nas arestas. Por exemplo, em C++, para representar o infinito utilize float a = std::numeric_limits<float>::infinity().
- 11) Fazer um método que permita remover um vértice do Grafo (não dirigido e dirigido). Não se esqueça de remover as arestas associadas.
- 12) Fazer um método que verifique e retorne se o grafo (não dirigido) é completo.
- 13) Fazer um método que verifique e retorne se o grafo (dirigido) é completo.
- 14) Fazer um método que retorne o complemento (grafo complementar) de um grafo (dirigido ou não) na forma de uma matriz de adjacência.
- 15) Fazer um método que retorne o tipo de conexidade de um grafo não direcionado (0 - conexo ou 1 - não conexo – desconexo).
- 16) Fazer um método que retorne a categoria de conexidade para um grafo direcionado (3 – C3, 2 – C2, 1 – C1 ou 0 – c0).
- 17) Fazer um método que retorne o grafo reduzido de um grafo direcionado no formato de uma matriz de adjacência.

Lista de Adjacência – Grafos Direcionados e Não direcionados

- 18) Escreva um método "int inDegree(int v)" que calcula e retorna o grau de entrada de um vértice v de um grafo dirigido fazendo uso da lista de adjacência.
- 19) Escreva o método outDegree(int v) que calcula o grau de saída de v em grafo dirigido. fazendo uso da lista de adjacência.
- 20) Fazer o método degree(int v) que calcula o grau do vértice de um grafo dirigido fazendo uso da lista de adjacência.
- 21) Escreva um método que decida se dois grafos direcionados são iguais. O método deve ser implementado para a classe TGrafo faz uso da lista de adjacência.
- 22) Escreva um método que converta uma representação de um grafo em outra. Por exemplo, converta um grafo armazenado em matriz de adjacência em uma lista de adjacência.
- 23) Escreva um método que receba um grafo armazenado em lista de adjacência e inverta a lista de adjacência de todos os vértices do grafo. Por exemplo, se os 4 vizinhos de um certo vértice **u** aparecem na lista **adj[u]** na ordem **v, w, x, y**, então depois da aplicação do método a lista deve conter os mesmos vértices na ordem **y, x, w, v**. Obs.: Vizinhos são todos os vértices ligados ao vértice u.
- 24) Escreva um método que receba um grafo e um vértice como parâmetro e retorne 1 se vértice for uma **fonte** (grau de saída maior que zero e grau de entrada igual a 0), ou 0 caso contrário. O método deve ser implementado para a classe TGrafo como lista de adjacência.
- 25) Escreva um método que receba um grafo e um vértice como parâmetro, retorne 1 se vértice for um **sorvedouro** (grau de entrada maior que zero e grau de saída igual a



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira



Teoria dos Grafos

- 0), ou 0 caso contrário. O método deve ser implementado para a classe TGrafo que utiliza lista de adjacência.
- 26) Escreva um método que receba um **grafo dirigido** como parâmetro e **retorna 1** se o grafo for **simétrico** e 0 caso contrário. O método deve ser implementado para a classe TGrafo que utiliza lista de adjacência.
- 27) Um **grafo** pode ser armazenado em um arquivo com o seguinte formato:

```
6
8
0 1
0 5
1 0
1 5
2 4
3 1
4 3
3 5
```

Onde na primeira linha contém um inteiro **V** (vértice), na segunda contém um inteiro **A** (arestas) e nas demais linha contém dois inteiros pertencentes ao intervalo 0..V-1. Se interpretarmos cada linha do arquivo como uma **aresta**, podemos dizer que o arquivo define um **grafo** com vértices 0..V-1. Escreva um método que receba um nome de arquivo com o formato acima e construa a representação de lista de adjacência do grafo.

- 28) Fazer um método que permita remover um vértice do Grafo (não dirigido). Não se esqueça de remover as arestas associadas.
- 29) Fazer um método que permita remover um vértice do Grafo (dirigido). Não se esqueça de remover as arestas associadas.
- 30) Fazer um método que verifique se o grafo (dirigido ou não) é completo.

O relatório a seguir deve conter o código fonte de cada um dos exercícios relacionados a implementação de Grafos representados como Matriz e Lista de Adjacência, incluindo um teste de cada um deles (printscreen).

Ao enviar pelo Ambiente Virtual, não se esquecer de INCLUIR o relatório e os códigos fontes de cada um deles na linguagem de programação utilizada para implementação (C++, C, Java ou Python) em um único arquivo compactado.

Obs.: Esta atividade tem o peso equivalente a 4 (quatro) atividades. A nota do grupo será proporcional a quantidade de exercícios implementados de forma correta.

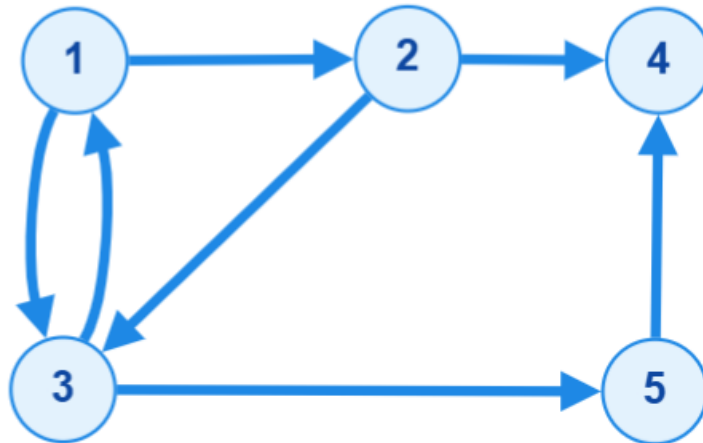


UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática
Prof. Dr. Ivan Carlos Alcântara de Oliveira
Teoria dos Grafos
Relatório

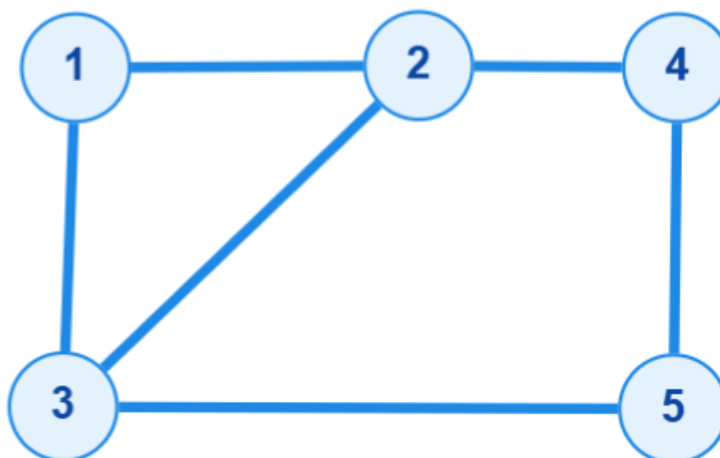


Grafos Usados como base para testes

A. Grafo Direcionado:

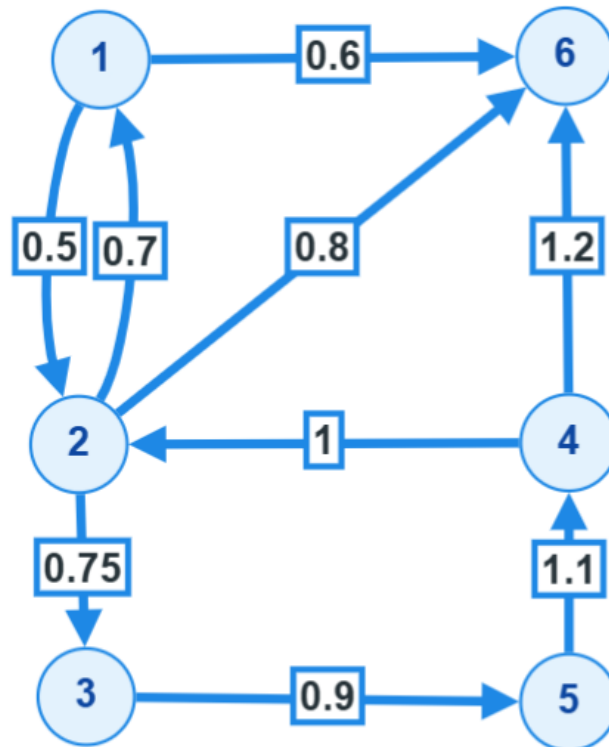


B. Grafo Não Direcionado:





C. Grafo Direcionado Ponderado:





Exercícios

1) Método "inDegree"

Código fonte:

```
public int inDegree(int v){  
    int degree = 0;  
    for(int i=0; i<n;i++){  
        if (adj[i][v].equals(1)){  
            degree += 1;  
        }  
    }  
    return degree;  
}
```

Teste:

```
Exercício 1- inDegree  
Grau de entrada do vértice 0: 1
```

2) Método "outDegree"

Código fonte:

```
public int outDegree(int v){  
    int degree = 0;  
    for(int i=0; i<n;i++){  
        if(adj[v][i].equals(1)){  
            degree += 1;  
        }  
    }  
    return degree;  
}
```

Teste:

```
Exercício 2- outDegree  
Grau de saída do vértice 0: 2
```



3) Método “degree”

Código fonte:

```
public int degree(int v){return this.inDegree(v) + this.outDegree(v);}
```

Teste:

```
Exercício 3- Degree  
Grau do vértice 0: 3
```

4) Método fonte

Código fonte:

```
public boolean nodeSource(int v){return this.outDegree(v) >= 1 &&  
this.inDegree(v) == 0;}
```

Teste:

```
Exercício 4- Fonte  
Vértice 0 é fonte? false
```

5) Método sorvedouro

Código fonte:

```
public boolean nodeReceiver(int v){return this.inDegree(v) >= 1 &&  
this.outDegree(v)==0;}
```

Teste:

```
Exercício 5- Sorvedouro  
Vértice 0 é sorvedouro? false
```



6) Método simétrico

Código fonte:

```
public boolean symmetric(){
    for(int i=0;i<n;i++){
        for(int j=0; j<n; j++){
            if(!Objects.equals(this.adj[i][j], this.adj[j][i])){
                return false;
            }
        }
    }
    return true;
}
```

Teste:

```
Exercício 6- Simétrico
Grafo é simétrico? false
```

7) Armazenar grafo em arquivo

Código fonte:

```
public class TGrafoMatrizBuilder {
    public static Grafo archiveReader(String archive) throws IOException {
        Path caminho = Path.of(archive);
        List<String> linhas = Files.readAllLines(caminho);

        Grafo gm = null;
        GraphType type = null;

        for (String linha : linhas) {
            if(Objects.equals(linha, linhas.getFirst())){
                if(linha.equalsIgnoreCase("ponderado")){
                    type = GraphType.PONDERADO;
                }
                else if(linha.equalsIgnoreCase("não ponderado")){type =
GraphType.NAO_PONDERADO;}
            }
            else if(Objects.equals(linha, linhas.get(1))){
                if (type != null) {
                    if(type == GraphType.NAO_PONDERADO){
                        gm = new TGrafoMatriz(Integer.parseInt(linha));
                    }
                    else {
                        gm = new TGrafoPonderadoMatriz(Integer.parseInt(linha));
                    }
                }
            }
        }
    }
}
```




```
    }
  }
  else{
    if(Objects.equals(linha, linhas.get(2))){
      continue;
    }
    String[] result = linha.split(" ");
    int v = Integer.parseInt(result[0]);
    int w = Integer.parseInt(result[1]);
    if (type != null && type.equals(GraphType.NAO_PONDERADO)){
      if (gm instanceof TGrafoMatriz aux){
        aux.insertA(v,w);
      }
    }
    else{
      float weight = Float.parseFloat(result[2]);
      if (gm instanceof TGrafoPonderadoMatriz aux){
        aux.insertA(v,w, weight);
      }
    }
  }
}
return gm;
}
```

Teste:

```
String arquivo = "Atividades/Lista_Implementação/Matriz.txt";
String arquivo1 = "Atividades/Lista_Implementação/ND.txt";
String arquivo2 = "Atividades/Lista_Implementação/MatrizPonderado.txt";
String arquivo3 = "Atividades/Lista_Implementação/ListaDirecionada.txt";

TGrafoListaND g_lista_nd = TGrafoListaNDBuilder.archiveReader(arquivo1);
TGrafoLista g_lista = TGrafoListaBuilder.archiveReader(arquivo3);
Grafo g = TGrafoMatrizBuilder.archiveReader(arquivo);
Grafo g_ponderado = TGrafoMatrizBuilder.archiveReader(arquivo2);
TGrafoNdMatriz g_nd = TGrafoNdMatrizBuilder.archiveReader(arquivo1);
```

```
Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1 Adj[0,3]= 0 Adj[0,4]= 0
Adj[1,0]= 0 Adj[1,1]= 0 Adj[1,2]= 1 Adj[1,3]= 1 Adj[1,4]= 0
Adj[2,0]= 1 Adj[2,1]= 0 Adj[2,2]= 0 Adj[2,3]= 0 Adj[2,4]= 1
Adj[3,0]= 0 Adj[3,1]= 0 Adj[3,2]= 0 Adj[3,3]= 0 Adj[3,4]= 0
Adj[4,0]= 0 Adj[4,1]= 0 Adj[4,2]= 0 Adj[4,3]= 1 Adj[4,4]= 0
```



8) Grafo não-dirigido com funções modificadas

Código fonte:

```
public void insertA(int v, int w) {
    if(adj[v][w].equals(0)){
        adj[v][w] = 1;
        arestas++;
    }
    else{
        System.out.println("Já existe uma aresta entre os nós " + v + " e " +
w);
    }
}

public void removeA(int v, int w) {
    if(adj[v][w].equals(1)){
        adj[v][w] = 0;
        arestas--;
    }
    else{
        System.out.println("Não existe uma aresta entre os nós " + v + " e " +
w);
    }
}

public void show() {
    System.out.println("n: " + n );
    System.out.println("m: " + arestas);
    for( int i=0; i < n; i++){
        System.out.print("\n");
        for( int w=0; w < n; w++){
            if(adj[i][w].equals(1)){
                System.out.print("Adj[" + i + "," + w + "]= 1" + " ");
            }
            else {System.out.print("Adj[" + i + "," + w + "]= 0" + " ");}
        }
        System.out.println("\n\nfim da impressão do grafo." );
    }
}
```



Teste:

Exercício 8- Grafo Não Direcionado

n: 5

m: 6

```
Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1 Adj[0,3]= 0 Adj[0,4]= 0
Adj[1,0]= 1 Adj[1,1]= 0 Adj[1,2]= 1 Adj[1,3]= 1 Adj[1,4]= 0
Adj[2,0]= 1 Adj[2,1]= 1 Adj[2,2]= 0 Adj[2,3]= 0 Adj[2,4]= 1
Adj[3,0]= 0 Adj[3,1]= 1 Adj[3,2]= 0 Adj[3,3]= 0 Adj[3,4]= 1
Adj[4,0]= 0 Adj[4,1]= 0 Adj[4,2]= 1 Adj[4,3]= 1 Adj[4,4]= 0
```

fim da impressao do grafo.

Remover aresta entre 0 e 2

n: 5

m: 5

```
Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 0 Adj[0,3]= 0 Adj[0,4]= 0
Adj[1,0]= 1 Adj[1,1]= 0 Adj[1,2]= 1 Adj[1,3]= 1 Adj[1,4]= 0
Adj[2,0]= 0 Adj[2,1]= 1 Adj[2,2]= 0 Adj[2,3]= 0 Adj[2,4]= 1
Adj[3,0]= 0 Adj[3,1]= 1 Adj[3,2]= 0 Adj[3,3]= 0 Adj[3,4]= 1
Adj[4,0]= 0 Adj[4,1]= 0 Adj[4,2]= 1 Adj[4,3]= 1 Adj[4,4]= 0
```

fim da impressao do grafo.



9) Método degree

Código fonte:

```
public int inDegree(int v){
    int degree = 0;
    for(int i=0; i<n;i++){
        if(adj[i][v] == 1){
            degree += 1;
        }
    }
    return degree;
}

public int degree(int v){return this.inDegree(v);}
```

Teste:

```
Exercício 9- Degree Não Direcionado
Grau do vértice 0: 1
```

10) Classe Grafo modificada para criação de grafo direcionado rotulado

Código fonte:

```
public interface Grafo {
    int inDegree(int v);
    int outDegree(int v);
    int degree(int v);
    boolean nodeSource(int v);
    boolean nodeReceiver(int v);
    boolean symmetric();
    boolean isComplete();
    void removeA(int v, int w);
    void show();
}

package Lista_Implementação.Exercicios_Matriz.Grafo_Direcionado.Grafo;

import java.util.Objects;

public class TGrafoPonderadoMatriz implements Grafo {
    private int n;
    private int arestas;
    private Float[][] adj;

    public TGrafoPonderadoMatriz(int n){
```



```
this.n = n;
this.arestas = 0;
this.adj = new Float[n][n];
for(int i = 0; i < n; i++){
    for(int j = 0; j < n; j++){
        this.adj[i][j] = Float.POSITIVE_INFINITY;
    }
}

@Override
public int inDegree(int v) {
    int degree = 0;
    for(int i=0; i<n;i++){
        if (!adj[i][v].equals(Float.POSITIVE_INFINITY)){
            degree += 1;
        }
    }
    return degree;
}

@Override
public int outDegree(int v) {
    int degree = 0;
    for(int i=0; i<n;i++){
        if(!adj[v][i].equals(Float.POSITIVE_INFINITY)){
            degree += 1;
        }
    }
    return degree;
}

@Override
public int degree(int v) {return this.inDegree(v) + this.outDegree(v);
}

@Override
public boolean nodeSource(int v) {return this.outDegree(v) >= 1 &&
this.inDegree(v) == 0;}

@Override
public boolean nodeReceiver(int v) {return this.inDegree(v) >= 1 &&
this.outDegree(v)==0;}

@Override
public boolean symmetric() {
    for(int i=0;i<n;i++){
        for(int j=0; j<n; j++){
```



```
        if(!Objects.equals(this.adj[i][j], this.adj[j][i])){
            return false;
        }
    }
}
return true;
}

@Override
public boolean isComplete() {
    return this.arestas == this.n*this.n-this.n;
}

public void insertA(int v, int w, float weight) {
    if(adj[v][w].equals(Float.POSITIVE_INFINITY)){
        adj[v][w] = weight;
        arestas++;
    }
    else{
        System.out.println("Já existe uma aresta entre os nós " + v + " e " + w
);
    }
}

@Override
public void removeA(int v, int w) {
    if(!adj[v][w].equals(Float.POSITIVE_INFINITY)){
        adj[v][w] = Float.POSITIVE_INFINITY;
        arestas--;
    }
    else{
        System.out.println("Não existe uma aresta entre os nós " + v + " e " +
w);
    }
}

@Override
public void show() {
    System.out.println("n: " + n );
    System.out.println("m: " + arestas);
    for( int i=0; i < n; i++){
        System.out.print("\n");
        for( int w=0; w < n; w++){
            if(!adj[i][w].equals(Float.POSITIVE_INFINITY)){
                System.out.print("Adj[" + i + ", " + w + "]= " + adj[i][w] + "
");
            }
        }
    }
}
```



```
    }  
    else {System.out.print("Adj[" + i + "," + w + "]= " +  
Float.POSITIVE_INFINITY + " ");}  
    }  
    System.out.println("\n\nfim da impressão do grafo." );  
}  
}
```

Teste:

Exercício 10- Grafo Ponderado Direcionado

n: 6

m: 9

```
Adj[0,0]= Infinity Adj[0,1]= 0.5 Adj[0,2]= Infinity Adj[0,3]= Infinity Adj[0,4]= Infinity Adj[0,5]= 0.6  
Adj[1,0]= 0.7 Adj[1,1]= Infinity Adj[1,2]= 0.75 Adj[1,3]= Infinity Adj[1,4]= Infinity Adj[1,5]= 0.8  
Adj[2,0]= Infinity Adj[2,1]= Infinity Adj[2,2]= Infinity Adj[2,3]= Infinity Adj[2,4]= 0.9 Adj[2,5]= Infinity  
Adj[3,0]= Infinity Adj[3,1]= 1.0 Adj[3,2]= Infinity Adj[3,3]= Infinity Adj[3,4]= Infinity Adj[3,5]= 1.2  
Adj[4,0]= Infinity Adj[4,1]= Infinity Adj[4,2]= Infinity Adj[4,3]= 1.1 Adj[4,4]= Infinity Adj[4,5]= Infinity  
Adj[5,0]= Infinity Adj[5,1]= Infinity Adj[5,2]= Infinity Adj[5,3]= Infinity Adj[5,4]= Infinity Adj[5,5]= Infinity
```

11) Método "removeV"

Código fonte grafo não dirigido:

```
public void removeV(int v){  
    Integer[][] aux = new Integer [n-1][n-1];  
    intiaux=0;  
    intjaux=0;  
    for(int i=0; i<this.n; i++){  
        for (int j=0; j<this.n; j++){  
            if(j!=v && i !=v){  
                aux[iaux][jaux] = this.adj[i][j];  
                jaux++;  
            }  
            else if (this.adj[i][j] != 0){  
                this.removeA(i,j);  
                this.removeA(j,i);  
            }  
        }  
        if(i != v){  
            iaux++;  
        }  
        jaux = 0;  
    }  
    this.n--;  
    this.adj = aux;  
}
```



Código fonte grafo dirigido:

```
public void removeV(int v){
    Integer[][] aux = new Integer [n-1][n-1];
    intiaux=0;
    intiaux=0;
    for(int i=0; i<this.n; i++){
        for (int j=0; j<this.n; j++){
            if(j!=v && i!=v){
                aux[iaux][iaux] = this.adj[i][j];
               iaux++;
            }
            else if(this.adj[i][j] != 0){
                this.removeA(i,j);
            }
        }
        if(i != v){
           iaux++;
        }
       iaux = 0;
    }
    this.n--;
    this.adj = aux;
}
```

Teste:

```
Exercício 11- Remover Vértice
Remover vértice 0 do grafo direcionado
n: 4
m: 4

Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1 Adj[0,3]= 0
Adj[1,0]= 0 Adj[1,1]= 0 Adj[1,2]= 0 Adj[1,3]= 1
Adj[2,0]= 0 Adj[2,1]= 0 Adj[2,2]= 0 Adj[2,3]= 0
Adj[3,0]= 0 Adj[3,1]= 0 Adj[3,2]= 1 Adj[3,3]= 0

fim da impressão do grafo.

Remover vértice 0 do grafo não direcionado
Não existe uma aresta entre os nós 1 e 0
n: 4
m: 4

Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1 Adj[0,3]= 0
Adj[1,0]= 1 Adj[1,1]= 0 Adj[1,2]= 0 Adj[1,3]= 1
Adj[2,0]= 1 Adj[2,1]= 0 Adj[2,2]= 0 Adj[2,3]= 1
Adj[3,0]= 0 Adj[3,1]= 1 Adj[3,2]= 1 Adj[3,3]= 0

fim da impressao do grafo.
```




12) Método “isComplete” para grafo não dirigido

Código fonte:

```
public boolean isComplete(){  
    return this.arestas == fatorial(this.n)/(fatorial(this.n - 2) *  
fatorial(2));  
}
```

Teste:

```
Exercício 12- Grafo Não Dirigido Completo  
O Grafo Não Dirigido é completo? false
```

13) Método “isComplete” para grafo dirigido

Código fonte:

```
public boolean isComplete(){return this.arestas == this.n*this.n-this.n;}
```

Teste:

```
Exercício 13- Grafo Dirigido Completo  
O Grafo Dirigido é completo? false
```

14) Método que retorna o complemento do grafo

Código fonte grafo não dirigido:

```
public TGrafoNdMatriz complement(){  
    TGrafoNdMatriz gaux = new TGrafoNdMatriz(this.n);  
    for(int i=0; i<this.n; i++){  
        for(int j=0; j<this.n; j++){  
            if(this.adj[i][j] == 0){  
                gaux.insertA(i, j);  
            }  
        }  
    }  
    return gaux;  
}
```

Código fonte grafo dirigido:

```
public TGrafoMatriz complement(){  
    TGrafoMatriz gaux = new TGrafoMatriz(this.n);  
    for(int i=0; i<this.n; i++){  
        for(int j=0; j<this.n; j++){  
            if(this.adj[i][j].equals(0)){  
                gaux.insertA(i, j);  
            }  
        }  
    }  
    return gaux;  
}
```



```
    }  
  }  
}  
return gaux;  
}
```

Teste:

Exercício 14- Complemento do Grafo

Complemento do Grafo Dirigido

n: 5

m: 18

Adj[0,0]= 1 Adj[0,1]= 0 Adj[0,2]= 0 Adj[0,3]= 1 Adj[0,4]= 1
Adj[1,0]= 1 Adj[1,1]= 1 Adj[1,2]= 0 Adj[1,3]= 0 Adj[1,4]= 1
Adj[2,0]= 0 Adj[2,1]= 1 Adj[2,2]= 1 Adj[2,3]= 1 Adj[2,4]= 0
Adj[3,0]= 1 Adj[3,1]= 1 Adj[3,2]= 1 Adj[3,3]= 1 Adj[3,4]= 1
Adj[4,0]= 1 Adj[4,1]= 1 Adj[4,2]= 1 Adj[4,3]= 0 Adj[4,4]= 1

fim da impressão do grafo.

Complemento do Grafo Não Dirigido

n: 5

m: 9

Adj[0,0]= 1 Adj[0,1]= 0 Adj[0,2]= 0 Adj[0,3]= 1 Adj[0,4]= 1
Adj[1,0]= 0 Adj[1,1]= 1 Adj[1,2]= 0 Adj[1,3]= 0 Adj[1,4]= 1
Adj[2,0]= 0 Adj[2,1]= 0 Adj[2,2]= 1 Adj[2,3]= 1 Adj[2,4]= 0
Adj[3,0]= 1 Adj[3,1]= 0 Adj[3,2]= 1 Adj[3,3]= 1 Adj[3,4]= 0
Adj[4,0]= 1 Adj[4,1]= 1 Adj[4,2]= 0 Adj[4,3]= 0 Adj[4,4]= 1

fim da impressao do grafo.



15) Método que retorna o tipo de conexidade de um grafo não direcionado

Código fonte:

```
public int conexidade(){
    List<Integer> visitados = this.largura(0);
    return (visitados.size() == n)?1:0;
}

public List<Integer> largura(int v){
    List<Integer> visitados = new ArrayList<>();
    Queue<Integer> fila = new LinkedList<>();

    visitados.add(v);
    fila.add(v);
    while (!fila.isEmpty()) {
        int n = fila.remove();
        for (int m = 0; m<this.n; m++) {
            if (this.adj[n][m] == 0) {
                continue;
            }
            if (!visitados.contains(m)) {
                fila.add(m);
                visitados.add(m);
            }
        }
    }
    return visitados;
}
```

Teste:

```
Exercício 15- Conexidade Grafo Não Dirigido
Conexidade: 1
```

16) Método que retorna o tipo de conexidade de um grafo direcionado

Código fonte:

```
public int conexidade(){
    Integer[][] fd = new Integer[this.n][this.n];
    Integer[][] fi = new Integer[this.n][this.n];
    for(int i=0; i<this.n; i++){
        List<Integer> aux = this.fechoTD(i);
        for (int j=0; j<this.n; j++) {
            if(aux.contains(j)){
                fd[i][j] = 1;
            }
        }
    }
}
```



```
        else{fd[i][j] = 0;}
    }
}

for(int i=0; i<this.n; i++){
    List<Integer> aux = this.fechoTI(i);
    for (int j=0;j<this.n;j++) {
        if(aux.contains(j)){
            fi[i][j] = 1;
        }
        else{fi[i][j] = 0;}
    }
}

//C3

boolean c3 = true;
for(int i=0;i<this.n;i++){
    for(int j=0;j<this.n;j++){
        if(fd[i][j]==1 && fi[i][j]==1){;
            continue;
        }
        else{
            c3 = false;
        }
    }
}

if(c3){return 3;}

//C2
boolean c2 = true;
Integer[][] union = new Integer[this.n][this.n];
for(int i=0;i<this.n;i++){
    for(int j=0;j<this.n;j++){
        if(fd[i][j] ==0 && fi[i][j]==0){
            c2 = false;
            union[i][j] = 0;
        }
        else{
            union[i][j] = 1;
        }
    }
}

if(c2){return 2;}
```



```
//C1
List<Integer> visitados = this.largura(0,union);
return (visitados.size() == n)?1:0;

}

public List<Integer> largura(int v, Integer[][]union){
    List<Integer> visitados = new ArrayList<>();
    Queue<Integer> fila = new LinkedList<>();

    visitados.add(v);
    fila.add(v);
    while (!fila.isEmpty()) {
        int n = fila.remove();
        for (int m = 0; m<this.n; m++) {
            if (union[n][m] == 0) {
                continue;
            }
            if (!visitados.contains(m)) {
                fila.add(m);
                visitados.add(m);
            }
        }
    }
    return visitados;
}

public List<Integer> fechoTD(int n){
    List<Integer> aux = new ArrayList<>();
    aux.add(n);
    return this.sucessor(n, aux);
}

public List<Integer> fechoTI(int n){
    List<Integer> aux = new ArrayList<>();
    aux.add(n);
    return this.antecessor(n, aux);
}

private List<Integer> sucessor(int v, List<Integer> aux){
    for (int i=0; i<this.n; i++){
        if(i==v) continue;
        if(this.adj[v][i] == 1 && !aux.contains(i)){
            aux.add(i);
            aux = sucessor(i, aux);
        }
    }
}
```



```
        return aux;
    }

    private List<Integer> antecessor(int v, List<Integer> aux){
        for (int i=0; i<this.n; i++){
            if(i==v) continue;
            if(this.adj[i][v] == 1 && !aux.contains(i)){
                aux.add(i);
                aux = antecessor(i, aux);
            }
        }
        return aux;
    }
}
```

Teste:

Exercício 16- Conexidade Grafo Dirigido
Conexidade: C2

17) Método que retorna o grafo reduzido

Código fonte:

```
public TGrafoMatriz grafoReduzido(){
    List<List<Integer>> ciclos = new ArrayList<>();
    List<Integer> acessados = new ArrayList<>();
    Map<Integer, Integer> vertice = new HashMap<>();

    for(int v=0; v<this.n; v++){
        if(acessados.contains(v)){continue;}
        List<Integer> fd = this.fechoTD(v);
        List<Integer> fi = this.fechoTI(v);
        fd.retainAll(fi);

        if(!fd.isEmpty()){
            ciclos.add(fd);
            int cicloIdx = ciclos.size() - 1;
            for (Integer ver : fd) {
                acessados.add(ver);
                vertice.put(ver, cicloIdx);
            }
        }
    }
}
```



Teoria dos Grafos

```
TGrafoMatriz grafoReduzido = new TGrafoMatriz(ciclos.size());
for(List<Integer> aux : ciclos){
    for(Integer i : aux){
        for(int j=0; j<this.n; j++){
            if(this.adj[i][j] == 1 && !aux.contains(j)){
                grafoReduzido.insertA(ciclos.indexOf(aux), vertice.get(j));
            }
        }
    }
}

return grafoReduzido;
}

public List<Integer> fechoTD(int n){
    List<Integer> aux = new ArrayList<>();
    aux.add(n);
    return this.sucessor(n, aux);
}

public List<Integer> fechoTI(int n){
    List<Integer> aux = new ArrayList<>();
    aux.add(n);
    return this.antecessor(n, aux);
}

private List<Integer> sucessor(int v, List<Integer> aux){
    for (int i=0; i<this.n; i++){
        if(i==v) continue;
        if(this.adj[v][i] == 1 && !aux.contains(i)){
            aux.add(i);
            aux = sucessor(i, aux);
        }
    }
    return aux;
}

private List<Integer> antecessor(int v, List<Integer> aux){
    for (int i=0; i<this.n; i++){
        if(i==v) continue;
        if(this.adj[i][v] == 1 && !aux.contains(i)){
            aux.add(i);
            aux = antecessor(i, aux);
        }
    }
    return aux;
}
```



Teste:

```
Exercício 17- Grafo Reduzido
n: 3
m: 3

Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1
Adj[1,0]= 0 Adj[1,1]= 0 Adj[1,2]= 0
Adj[2,0]= 0 Adj[2,1]= 1 Adj[2,2]= 0

fim da impressão do grafo.
```

18) Método “inDegree”

Código fonte:

```
public int inDegree(int v) {
    int qtde = 0;
    for (int i = 0; i < this.n; i++) {
        if (i != v){
            TNo aux = this.adj[i];
            while (aux != null) {
                if (aux.w == v) {
                    qtde++;
                    break;
                }
                aux = aux.prox;
            }
        }
    }

    return qtde;
}
```

Teste:

```
Exercício 18- inDegree Lista
Grau de entrada do vértice 0: 3
```




19) Método “inDegree”

Código fonte:

```
public int outDegree(int v) {  
    int qtde = 0;  
    TNo aux = this.adj[v];  
    while (aux != null) {  
        qtde++;  
        aux = aux.prox;  
    }  
  
    return qtde;  
}
```

Teste:

```
Exercício 19- outDegree Lista  
Grau de saída do vértice 0: 2
```

20) Método “degree”

Código fonte:

```
public int degree(int v) {return inDegree(v) + outDegree(v);}
```

Teste:

```
Exercício 20- Degree Lista  
Grau do vértice 0: 3
```

21) Método que verifica se dois grafos direcionados são iguais

Código fonte:

```
public boolean equal(TGrafoLista grafo){  
    if(this.n != grafo.getN() || grafo.getM() != this.m) return false;  
    TNo aux;  
    TNo aux_grafo;  
    for(int i=0; i<this.n; i++){  
        aux = this.adj[i];  
        aux_grafo = grafo.getAdj()[i];  
        while(aux != null && aux_grafo != null){  
            if(aux.w != aux_grafo.w){  
                return false;  
            }  
            aux = aux.prox;  
            aux_grafo = aux_grafo.prox;  
        }  
    }  
    return true;  
}
```



```
        aux_grafo = aux_grafo.prox;
    }
}

return true;
}
```

Teste:

```
Exercício 21- Grafo igual
Grafos são iguais: true
```

22) Método que transforma uma representação de grafo em outra

Código fonte:

```
public class Transformer {
    public TGrafoLista transformGrafoMatrizToLista(TGrafoMatriz gm) {
        TGrafoLista grafo = new TGrafoLista(gm.getN());

        for (int i = 0; i < gm.getN(); i++) {
            for (int j = 0; j < gm.getN(); j++) {
                if (gm.getAdj()[i][j] == 1) {
                    grafo.insereA(i, j);
                }
            }
        }

        return grafo;
    }

    public TGrafoMatriz transformGrafoListaToMatriz(TGrafoLista gm) {
        TGrafoMatriz grafo = new TGrafoMatriz(gm.getN());

        for (int i = 0; i < gm.getN(); i++) {
            TNo aux = gm.getAdj()[i];
            while(aux != null){
                int j = aux.w;
                grafo.insertA(i, j);
                aux = aux.prox;
            }
        }

        return grafo;
    }

    public TGrafoListaND transformGrafoMatrizToLista(TGrafoNdMatriz gm) {
```



```
TGrafoListaND grafo = new TGrafoListaND(gm.getN());

for (int i = 0; i < gm.getN(); i++) {
    for (int j = 0; j < gm.getN(); j++) {
        if (gm.getAdj()[i][j] == 1) {
            grafo.insereA(i, j);
        }
    }
}

return grafo;
}

public TGrafoNdMatriz transformGrafoListaToMatriz(TGrafoListaND gm) {
    TGrafoNdMatriz grafo = new TGrafoNdMatriz(gm.getN());

    for (int i = 0; i < gm.getN(); i++) {
        TNo aux = gm.getAdj()[i];
        while(aux != null){
            int j = aux.w;
            grafo.insertA(i, j);
            aux = aux.prox;
        }
    }

    return grafo;
}
```



Teste:

Exercício 22- Grafo Transformado

Grafo em Lista

n: 5

m: 7

0: 1 2

1: 2 3

2: 0 4

3:

4: 3

fim da impressao do grafo.

Transformado em Matriz

n: 5

m: 7

Adj[0,0]= 0 Adj[0,1]= 1 Adj[0,2]= 1 Adj[0,3]= 0 Adj[0,4]= 0

Adj[1,0]= 0 Adj[1,1]= 0 Adj[1,2]= 1 Adj[1,3]= 1 Adj[1,4]= 0

Adj[2,0]= 1 Adj[2,1]= 0 Adj[2,2]= 0 Adj[2,3]= 0 Adj[2,4]= 1

Adj[3,0]= 0 Adj[3,1]= 0 Adj[3,2]= 0 Adj[3,3]= 0 Adj[3,4]= 0

Adj[4,0]= 0 Adj[4,1]= 0 Adj[4,2]= 0 Adj[4,3]= 1 Adj[4,4]= 0

fim da impressão do grafo.

23) Método que inverte a lista

Código fonte:

```
public void inverteListas() {  
    for (int i = 0; i < this.n; i++) {  
        TNo atual = this.adj[i];  
        TNo ant = null;  
        TNo prox;  
  
        while (atual != null) {  
            prox = atual.prox;  
            atual.prox = ant;  
            ant = atual;  
            atual = prox;  
        }  
  
        this.adj[i] = ant;  
    }  
}
```



Teste:

Exercício 23- Inverter Lista

Grafo em Lista não invertido

n: 5

m: 7

0: 1 2

1: 2 3

2: 0 4

3:

4: 3

fim da impressao do grafo.

Grafo em Lista invertido

n: 5

m: 7

0: 2 1

1: 3 2

2: 4 0

3:

4: 3

fim da impressao do grafo.

24) Método fonte

Código fonte:

```
public boolean fonte(int v) {return (this.inDegree(v)==0 && this.outDegree(v) > 0);}
```

Teste:

Exercício 24- Fonte Lista

Vértice 0 é fonte? false



25) Método sorvedouro

Código fonte:

```
public boolean sorvedouro(int v) {return (this.inDegree(v)>0 &&
this.outDegree(v)==0);}
```

Teste:

```
Exercício 25- Sorvedouro Lista
Vértice 0 é sorvedouro? false
```

26) Método que verifica se o grafo é simétrico

Código fonte:

```
public boolean simetrico() {
    int[][] adj = new int[this.n][this.n];

    for (int i = 0; i<this.n; i++) {
        TNo aux = this.adj[i];
        while (aux != null) {
            int j = aux.w;
            adj[i][j] = 1;
            aux = aux.prox;
        }
    }

    for(int i=0;i<this.n;i++){
        for(int j=0; j<this.n; j++){
            if(!Objects.equals(adj[i][j], adj[j][i])){
                return false;
            }
        }
    }
    return true;
}
```

Teste:

```
Exercício 26- Simétrico Lista
Grafo é simétrico? false
```



27) Armazena o grafo em arquivo

Código fonte grafo direcionado:

```
public class TGrafoListaBuilder {
    public static TGrafoLista archiveReader(String archive) throws IOException {
        Path caminho = Path.of(archive);
        List<String> linhas = Files.readAllLines(caminho);

        TGrafoLista gm = null;

        for (String linha : linhas) {
            if(Objects.equals(linha, linhas.getFirst())){
                gm = new TGrafoLista(Integer.parseInt(linha));
            }
            else{
                if(Objects.equals(linha, linhas.get(1))){
                    continue;
                }
                String[] result = linha.split(" ");
                int v = Integer.parseInt(result[0]);
                int w = Integer.parseInt(result[1]);
                if (gm != null) {
                    gm.insereA(v,w);
                }
            }
        }
        return gm;
    }
}
```

Código fonte grafo não direcionado:

```
public class TGrafoListaNDBuilder {
    public static TGrafoListaND archiveReader(String archive) throws IOException {
        Path caminho = Path.of(archive);
        List<String> linhas = Files.readAllLines(caminho);

        TGrafoListaND gm = null;

        for (String linha : linhas) {
            if(Objects.equals(linha, linhas.getFirst())){
                gm = new TGrafoListaND(Integer.parseInt(linha));
            }
            else{
                if(Objects.equals(linha, linhas.get(1))){
                    continue;
                }
            }
        }
    }
}
```



Teoria dos Grafos

```
String[] result = linha.split(" ");
int v = Integer.parseInt(result[0]);
int w = Integer.parseInt(result[1]);
if (gm != null) {
    gm.insereA(v,w);
}
}
}
return gm;
}
```

Teste:

```
String arquivo = "Atividades/Lista_Implementação/Matriz.txt";
String arquivo1 = "Atividades/Lista_Implementação/ND.txt";
String arquivo2 = "Atividades/Lista_Implementação/MatrizPonderado.txt";
String arquivo3 = "Atividades/Lista_Implementação/ListaDirecionada.txt";

TGrafoListaND g_lista_nd = TGrafoListaNDBuilder.archiveReader(arquivo1);
TGrafoLista g_lista = TGrafoListaBuilder.archiveReader(arquivo3);
Grafo g = TGrafoMatrizBuilder.archiveReader(arquivo);
Grafo g_ponderado = TGrafoMatrizBuilder.archiveReader(arquivo2);
TGrafoNdMatriz g_nd = TGrafoNdMatrizBuilder.archiveReader(arquivo1);
```

Exercício 27- Ler Arquivo Lista

n: 5

m: 7

0: 1 2

1: 2 3

2: 0 4

3:

4: 3

fim da impressao do grafo.

n: 5

m: 12

0: 1 2

1: 0 2 3

2: 0 1 4

3: 1 4

4: 2 3



Teoria dos Grafos

28) Método que remove vértice do grafo não dirigido

Código fonte:

```
public void removeV(int v) {  
    while (adj[v] != null) {  
        removeA(v, adj[v].w); // já remove v↔w  
    }  
  
    for (int i = v; i < n - 1; i++) {  
        adj[i] = adj[i + 1];  
    }  
    adj[n - 1] = null;  
  
    for (int i = 0; i < n - 1; i++) {  
        TNo aux = adj[i];  
        while (aux != null) {  
            if (aux.w > v) {  
                aux.w--;  
            }  
            aux = aux.prox;  
        }  
    }  
  
    n--;  
}
```

Teste:

Exercício 28- Remover Vértice Lista Não Direcionada

Grafo em Lista Antes da Remoção

n: 5
m: 12

0: 1 2
1: 0 2 3
2: 0 1 4
3: 1 4
4: 2 3

fim da impressao do grafo.

Grafo em Lista Depois da Remoção do Vértice 0

n: 4
m: 8

0: 1 2
1: 0 3
2: 0 3
3: 1 2

fim da impressao do grafo.



29) Método que remove vértice do grafo dirigido

Código fonte:

```
public void removeV(int v) {
    while (adj[v] != null) {
        removeA(v, adj[v].w);
    }

    for (int i = 0; i < n; i++) {
        if (i != v) {
            removeA(i, v);
        }
    }

    for (int i = v; i < n - 1; i++) {
        adj[i] = adj[i + 1];
    }
    adj[n - 1] = null;

    for (int i = 0; i < n - 1; i++) {
        TNo aux = adj[i];
        while (aux != null) {
            if (aux.w > v) {
                aux.w--;
            }
            aux = aux.prox;
        }
    }

    n--;
}
```



Teste:

```
Exercício 29- Remover Vértice Lista Direcionada
```

```
Grafo em Lista Antes da Remoção
```

```
n: 5
```

```
m: 7
```

```
0: 1 2
```

```
1: 2 3
```

```
2: 0 4
```

```
3:
```

```
4: 3
```

```
fim da impressao do grafo.
```

```
Grafo em Lista Depois da Remoção do Vértice 0
```

```
n: 4
```

```
m: 4
```

```
0: 1 2
```

```
1: 3
```

```
2:
```

```
3: 2
```

```
fim da impressao do grafo.
```



30) Método que verifica se o grafo é completo

Código fonte:

Direcionado:

```
public boolean isComplete(){return this.m == this.n*this.n-this.n;}
```

Não direcionado:

```
public boolean isComplete(){  
    return this.m == fatorial(this.n)/(fatorial(this.n - 2) * fatorial(2));  
}
```

Teste:

```
Exercício 30- Grafo Completo Lista  
O Grafo em Lista Não Dirigido é completo? false  
O Grafo em Lista Dirigido é completo? false
```